Cognizant - UpGrad Training Program
WEEKLY CODING SUBMISSION

Name	**:**	**Durgesh Vijay Bhangale**
Week	**:**	**Week 7**
Batch	**:**	**B1_Python**
Trainer	**:**	**Kunal**

Day 1(30-3-26)

Q1: Extensible Notification System with Failure Handling

```
public interface INotificationSender {
   Task SendAsync(string message);
}

public class EmailSender : INotificationSender {
   public async Task SendAsync(string message) {
      try {
         Console.WriteLine($"Sending Email: {message}");

      } catch {
         Console.WriteLine("Email failed, retrying...");
         try { /* retry */ }
         catch { Console.WriteLine("Email permanently failed"); }
      }
   }
}

public class SmsSender : INotificationSender {
   public async Task SendAsync(string message) {
      try {
         Console.WriteLine($"Sending SMS: {message}");

         throw new Exception("SMS timeout");
      } catch {
         Console.WriteLine("SMS failed, retrying...");
         try { /* retry */ }
         catch { Console.WriteLine("SMS permanently failed"); }
      }
   }
}

public class NotificationService {
   private readonly IEnumerable<INotificationSender> _senders;
   public NotificationService(IEnumerable<INotificationSender> senders) {
      _senders = senders;
   }

   public async Task SendBulkAsync(string message) {
      var tasks = _senders.Select(s => s.SendAsync(message));
      await Task.WhenAll(tasks);
   }
}
```

Q2: Thread-Safe In-Memory Cache with Expiration Bug

```
public sealed class InMemoryCache {
    private static readonly Lazy<InMemoryCache> _instance = new(() => new InMemoryCache());
    public static InMemoryCache Instance => _instance.Value;

    private readonly ConcurrentDictionary<string,(object val, DateTime expiry)> _store = new();

    private InMemoryCache() {
        Task.Run(async () => {
            while (true) {
                foreach (var kv in _store) {
                    if (kv.Value.expiry < DateTime.UtcNow) {
                        _store.TryRemove(kv.Key, out _);
                    }
                }
                await Task.Delay(1000);
            }
        });
    }

    public void Set(string key, object value, TimeSpan ttl) =>
        _store[key] = (value, DateTime.UtcNow.Add(ttl));

    public object? Get(string key) =>
        _store.TryGetValue(key, out var item) && item.expiry > DateTime.UtcNow ? item.val : null;
}
```

Q3: JWT Authentication System with Random Failures

```
builder.Services.AddAuthentication("Bearer")
    .AddJwtBearer(opt => {
        opt.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = "MyApp",
            ValidAudience = "MyAppUsers",
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes("supersecretkey")),
            ClockSkew = TimeSpan.FromMinutes(1)
        };
    });

app.UseAuthentication();
app.UseAuthorization();
```

Q4: API Rate Limiting System (Scalable Design)

```
public class RateLimiter {
    private readonly ConcurrentDictionary<string, Queue<DateTime>> _requests = new();

    public bool Allow(string userId) {
        var now = DateTime.UtcNow;
        var q = _requests.GetOrAdd(userId, _ => new Queue<DateTime>());
        lock (q) {
            while (q.Count > 0 && (now - q.Peek()).TotalSeconds > 10) q.Dequeue();
            if (q.Count >= 5) return false;
            q.Enqueue(now);
```

```
            return true;
        }
    }
}
```

Q5: Deadlock Creation and Resolution

```
object lockA = new(), lockB = new();

void Deadlock() {
    lock(lockA) {
        Thread.Sleep(100);
        lock(lockB) { Console.WriteLine("Work done"); }
    }
}

void Fixed() {
    lock(lockA) {
        lock(lockB) { Console.WriteLine("Work done safely"); }
    }
}
```

Q6: Custom HashMap with Performance Degradation

```
public class MyHashMap<K,V> {
    private List<(K key,V val)>[] buckets;
    private int count;
    private const double LoadFactor = 0.75;

    public MyHashMap(int size=16) => buckets = new List<(K,V)>[size];

    private int Hash(K key) => Math.Abs(key!.GetHashCode()) % buckets.Length;
```

```
    public void Add(K key, V val) {
        if (count > buckets.Length * LoadFactor) Resize();
        var idx = Hash(key);
        buckets[idx] ??= new List<(K,V)>();
        buckets[idx].Add((key,val));
        count++;
    }

    public V? Get(K key) => buckets[Hash(key)]?.Find(p => p.key!.Equals(key)).val;

    private void Resize() {
        var newBuckets = new List<(K,V)>[buckets.Length*2];
        foreach (var bucket in buckets) {
            if (bucket == null) continue;
            foreach (var (k,v) in bucket) {
                var idx = Math.Abs(k!.GetHashCode()) % newBuckets.Length;
                newBuckets[idx] ??= new List<(K,V)>();
                newBuckets[idx].Add((k,v));
            }
        }
        buckets = newBuckets;
    }
}
```

Q7: High-Load Order Processing System (ALL TOPICS COMBINED)

```
public class OrderService {
    private readonly IOrderRepo _repo;
    private readonly ICache _cache;
    private readonly IEnumerable<INotificationSender> _senders;
```

```
    public OrderService(IOrderRepo repo, ICache cache, IEnumerable<INotificationSender> senders) {
        _repo = repo; _cache = cache; _senders = senders;
    }

    public async Task PlaceOrder(Order o) {
        if (_cache.Get(o.Id.ToString()) != null) return;

        await _repo.Save(o);
        _cache.Set(o.Id.ToString(), o, TimeSpan.FromMinutes(5));

        var tasks = _senders.Select(s => s.SendAsync($"Order {o.Id} confirmed"));
        await Task.WhenAll(tasks);
    }

    public Order? GetOrder(string id) {
        return (Order?)_cache.Get(id) ?? _repo.GetById(id);
    }
}
```

Day 2(31-3-26)

PROJECT 2:

“Personal Productivity Heatmap Dashboard”

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Productivity Tracker</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>My Productivity Tracker</h1>

  <div id="metrics">
    <div class="metric">Active Days: <span id="activeDays">0</span></div>
    <div class="metric">Total Hours: <span id="totalHours">0</span></div>
    <div class="metric">Tasks Done: <span id="totalTasks">0</span></div>
    <div class="metric">Streak: <span id="streak">0</span></div>
  </div>

  <h2>2026 Activity</h2>
  <div id="heatmap"></div>

  <div id="tooltip" class="tooltip"></div>

  <h2>Log Activity</h2>
  <p>Click a cell to select a date</p>
  <label>Hours Studied:</label>
  <input type="number" id="hours" min="0" placeholder="e.g. 3">
  <label>Tasks Completed:</label>
```

```
  <input type="number" id="tasks" min="0" placeholder="e.g. 5">
  <button id="saveBtn">Save</button>

  <script src="script.js"></script>
</body>
</html>
```

Style.css

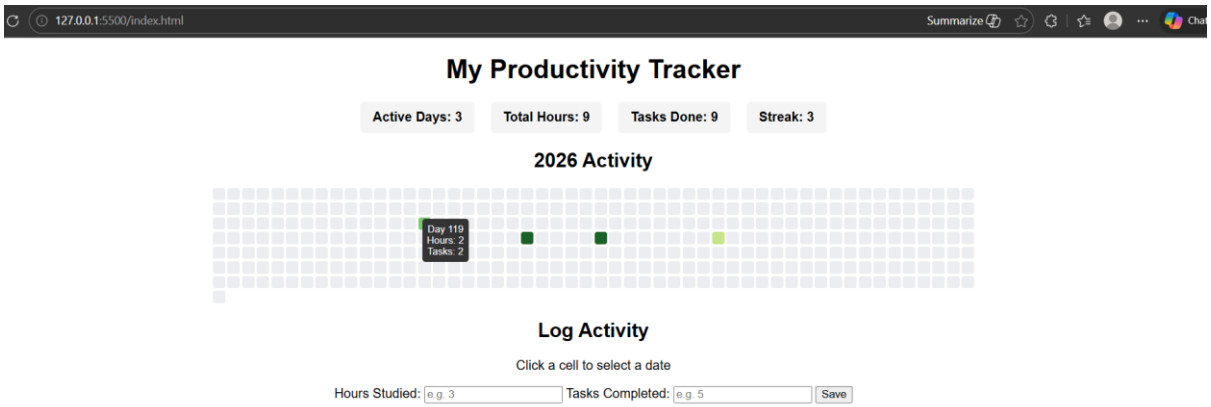
```
body {
  font-family: Arial, sans-serif;
  text-align: center;
  background: #fff;
}

#metrics {
  display: flex;
  justify-content: center;
  gap: 20px;
  margin: 20px;
}

.metric {
  background: #f4f4f4;
  padding: 10px 15px;
  border-radius: 6px;
  font-weight: bold;
}

#heatmap {
  display: grid;
  grid-template-columns: repeat(52, 1fr); /* 52 weeks */
```

```
  grid-template-rows: repeat(7, 1fr);     /* 7 days */
  gap: 3px;
  width: fit-content;
  margin: 20px auto;
}

.cell {
  width: 15px;
  height: 15px;
  border-radius: 3px;
  background: #ebedf0;
  transition: transform 0.2s;
}

.cell:hover {
  transform: scale(1.3);
  cursor: pointer;
}

.low { background: #c6e48b; }
.medium { background: #7bc96f; }
.high { background: #196127; }

.tooltip {
  position: absolute;
  background: #333;
  color: #fff;
  padding: 6px;
  border-radius: 4px;
  display: none;
  font-size: 12px;
```

```
}
```

Script.js

```
const heatmap = document.getElementById("heatmap");
const tooltip = document.getElementById("tooltip");
const saveBtn = document.getElementById("saveBtn");
let currentDay = null;

for (let i = 1; i <= 365; i++) {
  let cell = document.createElement("div");
  cell.classList.add("cell");
  cell.dataset.day = i;

  let saved = localStorage.getItem(i);
  if (saved) applyColor(cell, JSON.parse(saved).hours);

  cell.addEventListener("click", () => {
    currentDay = i;
    document.getElementById("hours").value = "";
    document.getElementById("tasks").value = "";
  });

  cell.addEventListener("mouseover", (e) => {
    let data = localStorage.getItem(i);
    if (data) {
      let obj = JSON.parse(data);
      tooltip.innerText = `Day ${i}\nHours: ${obj.hours}\nTasks: ${obj.tasks}`;
      tooltip.style.display = "block";
      tooltip.style.left = e.pageX + "px";
      tooltip.style.top = e.pageY + "px";
    }
```

```
  });
  cell.addEventListener("mouseout", () => tooltip.style.display = "none");

  heatmap.appendChild(cell);
}

saveBtn.onclick = () => {
  if (!currentDay) return alert("Select a day first!");
  let hours = parseInt(document.getElementById("hours").value) || 0;
  let tasks = parseInt(document.getElementById("tasks").value) || 0;
  localStorage.setItem(currentDay, JSON.stringify({hours, tasks}));

  let cell = document.querySelector(`[data-day='${currentDay}']`);
  applyColor(cell, hours);

  updateMetrics();
};

function applyColor(cell, hours) {
  cell.className = "cell";
  if (hours === 0) cell.classList.add("low");
  else if (hours < 3) cell.classList.add("medium");
  else cell.classList.add("high");
}

function updateMetrics() {
  let activeDays = 0, totalHours = 0, totalTasks = 0, streak = 0, currentStreak = 0;
  for (let i = 1; i <= 365; i++) {
    let data = localStorage.getItem(i);
    if (data) {
      let obj = JSON.parse(data);
```

```
    if (obj.hours > 0 || obj.tasks > 0) {
      activeDays++;
      totalHours += obj.hours;
      totalTasks += obj.tasks;
      currentStreak++;
      streak = Math.max(streak, currentStreak);
    } else {
      currentStreak = 0;
    }
  }
}
document.getElementById("activeDays").innerText = activeDays;
document.getElementById("totalHours").innerText = totalHours;
document.getElementById("totalTasks").innerText = totalTasks;
document.getElementById("streak").innerText = streak;
}

updateMetrics();
```